

Implementation of Big Data Architecture for E-Commerce Analytics and Customer Satisfaction Prediction on Brazilian Olist Data



Presented by:
Group 3



Phase 1

Business & Big Data Justification



Presented by:
Group 3

Bussiness Domain

E-commerce is a sector of businesses that offers the wholesale sale of goods through a digital platform or marketplace that brings buyers together with sellers.

Bussiness Entities

Sellers & Consumers

The main pillar that drives supply and demand flow of goods dynamically within the marketplace.

Payment & Logistics

Supporting infrastructure that ensures payment transaction security and timeliness

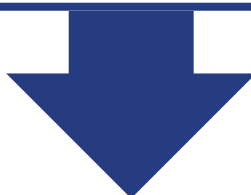
Customer Reviews

A transparent feedback system of buyer once goods are received, serves building trust ecosystem.



May used to understand :

- consumer behavior
- sales performance
- item sales patterns
- effectiveness of delivery process



Support Bussiness Descision Making



Generate Vast Amount of Data



Problem Statement

The Challenge (Reality of The Data)

- **Data Potential:** Transactional data holds critical information regarding business activities.
- **Risk:** **Improperly conducted analytics** makes it difficult to understand customer behavior, product performance, and operational efficiency.
- **Impact:** Valuable insights from dataset are not optimally exploited.

Data Exploration Focus



Products & Trend

- best-selling items
- sales trends over time

guide inventory and promotional strategies



Customer Insight

- purchase behaviour
- customer reviews

evaluate satisfaction and rating factors



Operations

- seller performance
- logistic performance

improve user experience

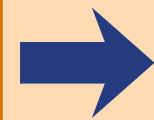
Solution & Value Creation

Tech Leverage

Effective Big Data processing

Strategic Insights

Data-driven decision making



Final Value

Enhanced customer experience

Big Data Characteristics

1. Volume

Volume refers to the amount or size of data that must be stored and processed. The Olist Brazil E-Commerce dataset consists of various interconnected activity tables with the following estimated data volumes:

Orders	±100.000 orders data
Order Items	±110.000 product item bought
Customers	±99.000 customers
Payments	±100.000 payment transaction
Reviews	±100.000 customer reviews
Products	±30.000 products

2. Velocity

Velocity refers to the speed at which data is generated, enters the system, and is processed.

- **Real-Time Data Flow:** In a real-world Olist system, data flows continuously from user activities (checkout processes, automatic payment recording, logistics status updates by couriers, and review submissions).
- **Daily Scale:** The system generates thousands of transactions per day that must be processed simultaneously.

Big Data Characteristics

3. Variety

Variety refers to the diversity of data types contained in a big data system. In the Olist dataset, the data consists of structured, and unstructured formats. Structured data appears in transaction tables such as orders and payments that already have fixed rows and columns. Unstructured data can be found in customer review comments because the text is written freely by users. Meanwhile, unstructured formats appear in the `olist_review_comment` where the row is filled with comment text from customer. This diversity makes data integration and processing more complex compared to traditional datasets.

4. Veracity

Veracity represents the quality, consistency, and reliability of the data. The Olist dataset contains several data quality issues such as missing values, duplicate records, inconsistent delivery information, and financial outliers.. These problems can affect analytical results if not handled properly. Therefore, data cleaning and preprocessing become important steps before performing analytics and machine learning.

5. Value

Value refers to the business benefits that can be extracted from the data after processing and analysis. From the Olist dataset, companies can gain insights related to customer satisfaction, delivery performance, product demand, and seller performance. The analysis can help businesses optimize logistics, improve customer experience, predict sales trends, and support data-driven decision making. This shows that large amounts of data become meaningful only when they can generate actionable business insights.

Business Motivation

Business Background

Big data analytics helps companies understand customer behavior, sales activities, and operational performance within e-commerce platforms.

• Rich Transactional Data

The dataset contains various types of e-commerce transaction data, including orders, payments, shipping information, and customer reviews. These data sources provide valuable insights into customer behavior and operational activities within the platform.

- Large-scale relational transaction dataset
- Supports predictive analytics and business insights

• Data-Driven Growth

Big data analytics helps companies identify customer purchasing patterns, evaluate logistics performance, and support strategic business decisions. The generated insights can improve customer satisfaction and increase company profitability.

- Enables faster operational decision-making
- Supports customer satisfaction prediction



Reasons for This Problem Considered as Big Data Issues

○ **Volume (Large Amount of Data)**

Dataset contains millions of transaction records such as orders, customers, products, payments, and reviews, requiring big data technology for efficient processing.

○ **Variety (Different Types of Data)**

Data consists of structured, semi structured, and unstructured data from multiple tables like transactions, customers, sellers, products, payments, and reviews.

○ **Velocity (Rapid Data Growth)**

E-commerce transactions occur continuously every day, causing data to grow rapidly and requiring fast data processing for real-time insights.

○ **Value (Business Value of Data)**

The data provides valuable insights for analyzing customer behavior, popular products, seller performance, and improving service quality.

Limitations of Traditional Data Processing Approaches



○ Vertical Scalability Issues

- **Traditional Approach:** Relies on upgrading a single server's RAM/CPU.
- **The Bottleneck:** Hits a physical limit and becomes extremely expensive.
- **Big Data Solution:** Uses horizontal scaling by adding cheap commodity hardware to a cluster.

○ Rigid Data Schema

- **Traditional Approach:** Requires data to fit into strict, predefined table schemas before saving.
- **The Bottleneck:** Inflexible and fails to handle unstructured data.
- **The Challenge:** Struggles with dynamic formats like social media feeds and logs.

Limitations of Traditional Data Processing Approaches



○ Limited Processing Speed

- **Traditional Approach:** Processes data sequentially (one after another).
- **The Bottleneck:** Query speeds drop drastically at Terabyte or Petabyte scale.
- **Big Data Solution:** Uses parallel and distributed processing across multiple nodes simultaneously.

○ Infrastructure Cost Burden

- **Traditional Approach:** Demands high investment for software licenses and dedicated servers.
- **The Bottleneck:** Scaling up traditional enterprise infrastructure is financially unsustainable.
- **Big Data Solution:** Highly cost-efficient due to open-source software running on standard hardware.



Phase 2

Big Data Architecture Design



Presented by:
Group 3

Architecture Overview

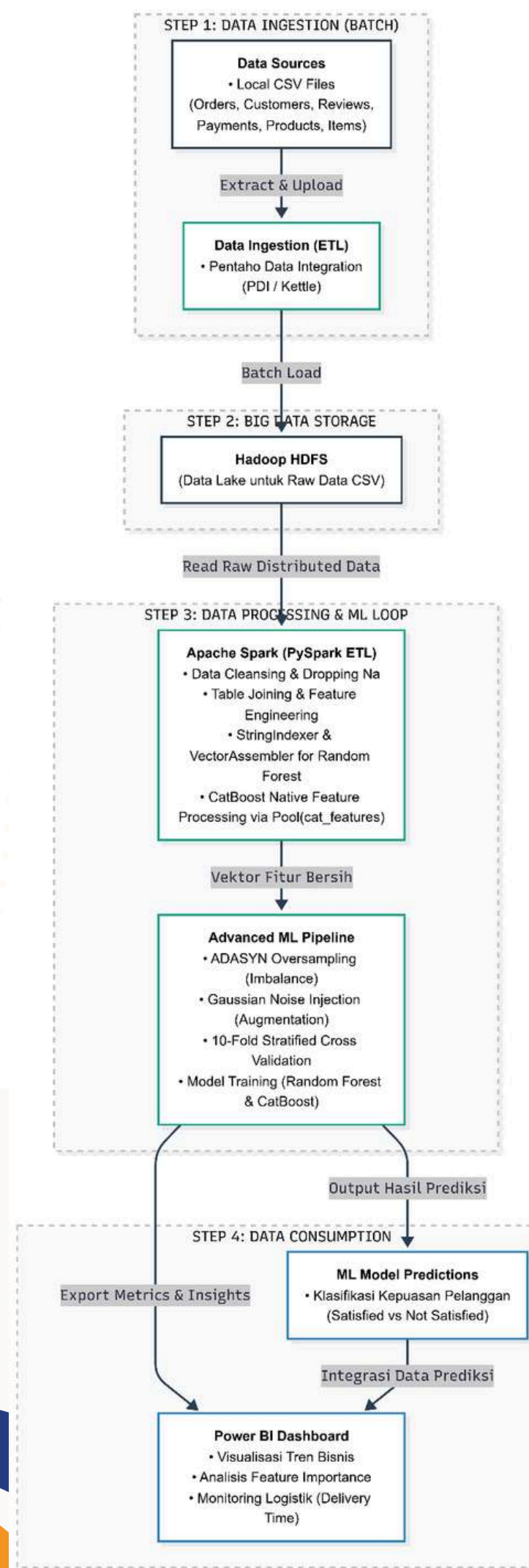
This architecture is designed to process and analyze the Brazilian Olist e-commerce dataset that has 5V characteristics (Volume, Velocity, Variety, Veracity, and Value).

Several main layers from this architecture:

- **Data Sources:** Collects raw data of e-commerce transactions in CSV format.
- **Ingestion Layer:** Performs the process of loading data from source to storage using the Batch ETL pattern.
- **Storage Layer:** Responsible for storing distributed data using HDFS as primary storage for large volumes.
- **Processing & Analytics Engine:** Uses Apache Spark as the primary processing engine to run transformations, analytics, and machine learning (customer satisfaction classification) models.
- **Visualization Layer:** Convert analytics results into easy-to-understand visualizations, such as interactive dashboards, using tools like Power BI.

Data Source

Data is retrieved from CSV format in Brazilian Olist e-commerce dataset which this is include groups of structured data.



Ingestion Layer

What is Data Ingestion?

Process of loading data from one or multiple sources into storage (cloud or on-premise) for further processing, analysis, and modeling .

Ingestion Methods

Batch

- Large batches at specific intervals
- Historical data
- No real-time need

Streaming

- Continuous as data is generated
- Real-time analytics
- Fraud detection

Hybrid

- Combination if both methods
- Flexible approach
- Mixed use cases

Ingestion Pattern

Extract

Transform

Load

VS

Extract

Load

Transform

Ingestion layer Design for Brazilian Online Shop Dataset

Volume

Thousands to millions of transaction

Velocity

Stable speed, processed in batches (not real-time)

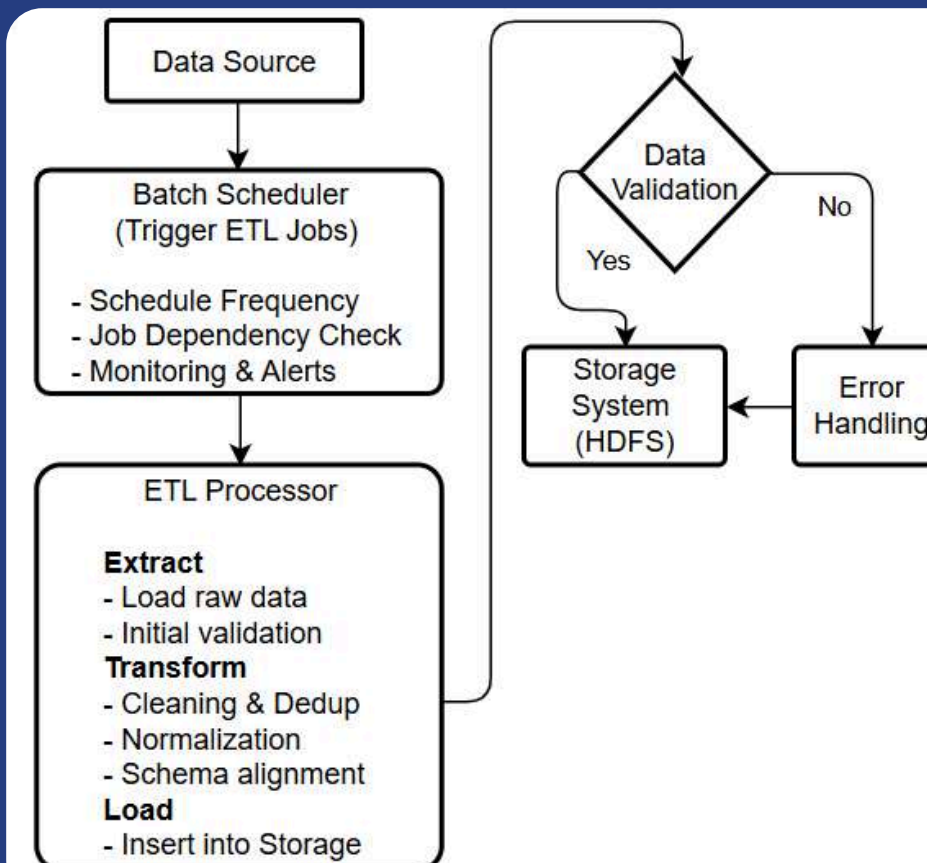
Variety

Multiple data types needs standardization

Tools and frequency

Python + Pandas (CSV). Structured DB / DWH. One-time batch load, static dataset, no repetitive processing needed.

Data Ingestion and ETL Processing Workflow



Data Validation

- 1.record count check
- 2.Data Structure Appropriateness
- 3.Value Consistency



Storage Layer

○ HDFS as Primary Storage

- Large-Scale Capacity: Efficiently manages millions of rows across 9 dataset tables.
- Batch Processing Optimized: Delivers the high throughput required for non-real-time analytical workflows.
- Data Versatility: Easily accommodates both structured and semi-structured data types.

○ Key Architectural Advantages

- Horizontal Scalability: Flexible and cost-effective cluster expansion using standard commodity hardware.
- Fault Tolerance: Automatically breaks data into 128 MB blocks and replicates them across multiple servers.
- High Reliability: Built-in duplication ensures data remains completely safe and accessible even if a server fails.





Processing & Analytics Engine



The Processing Engine (Data Handling)

you can think of it as the "brain" of the entire system. It is where the raw data stored in HDFS is actually put to work.

- **Distributed Power:** Unlike traditional systems that process data sequentially on one server, this engine distributes the workload across a cluster of machines to handle massive scales.
- **Apache Spark as the Workhorse:** Spark is the primary tool used because it excels at high-performance, distributed processing.
- **Batch Processing Choice:** Since the Brazilian E-Commerce dataset is historical and static, the engine uses a batch approach rather than real-time processing, which is more efficient for this specific use case.
- **Flexible Tooling:** Python and the Pandas library are used during the processing stages to allow for flexible reading and manipulation of the CSV data.



Processing & Analytics Engine



The Analytics & Machine Learning Engine (Advanced Insights)

- **Predictive Goal:** The primary objective is to build a classification model that predicts Customer Satisfaction.
- **Binary Classification:** The engine simplifies satisfaction into two classes: "Satisfied" (scores > 4) and "Not Satisfied" (scores < 4).
Model Selection (CatBoost): CatBoost was selected as the main algorithm because it handles categorical data (like cities or states) automatically and includes "Ordered Boosting" to prevent the model from memorizing the data instead of learning from it.
- **Validation Rigor:** The engine uses a 10-fold cross-validation method, meaning it trains and tests the model 10 different times to ensure the results are stable and unbiased.



Visualization Layer

Key Roles & Objectives

- **Data Visualization:** Transforming complex technical analysis results into visual formats (charts, dashboards, reports) that are easy for business operations teams to understand.
- **Decision-Making:** Enhancing the effectiveness of data-driven decision-making by aligning data insights with business needs.

Focus on Data Analysis & Optimization

- **Data Format:** Use the Parquet format to structure data efficiently and filter out unnecessary details (for visualization purposes only).
- 3 Key Insights:
 1. **Geographic Insights:** Sales distribution by region.
 2. **Category Impact:** Identify products with high sales volume.
 3. **Seasonal Patterns:** Observe cyclical/seasonal sales trends.

Infrastructure & Tools Used

- **Power BI:** Chosen for its efficiency in generating interactive dashboards, built-in compatibility with data outputs, and its filtering and drill-down features for in-depth data analysis. Batch
- **Visualization Method:** Since this Brazilian e-commerce data is static (does not require real-time processing), this project uses a batch method, which is far more efficient:
Workflow: Data is preprocessed by Apache Spark → Results are stored → Visualizations are updated periodically. (Without the need to build complex streaming infrastructure).

Monitoring & Security

○ Pipeline & Ingestion Monitoring

Tracking data flows from ingestion to processing using active logging and instant failure notifications.

- Strategic Goal:
Ensuring high data availability and detecting system bottlenecks before they degrade processing performance.

○ Data Security & Access Control

- Securing sensitive customer transaction history with robust Kerberos-based authentication.
 - Strategic Goal:
- Restricting cluster access to authorized users and logging anomalies (e.g., failed login attempts, illegal access) for security auditing.

○ Storage Infrastructure Monitoring (HDFS Stability)

tracking of node health, storage capacity, and data replication using tools like Apache Ambari and Grafana.

Strategic Goal:

Ensuring high data availability and detecting system bottlenecks before they degrade processing performance.



Infrastructure Decision Analysis

○ Project Infrastructure

- Basis: High-volume Olist E-Commerce dataset.
- Architecture: Distributed Big Data (replacing single-node).
- Environment: VM-based for flexible, low-cost deployment.

○ Storage & Processing

- Storage: HDFS for distributed data consistency.
- Mode: Batch processing for historical records.
- Engine: Apache Spark for parallel ML & transformations.
- Prep: Python & Pandas for initial CSV cleaning.

○ Visualization & Result

- Analytics: Power BI interactive dashboards.
- Outcome: Optimized workflow for customer satisfaction prediction.





Phase 3 Implementation

```
source_id' => $resource_id  
if ( ! $resource_exists( $resource_details['id'] ) ) {  
    $access = false }  
// Remove the rule as there is currently no resource  
$this->sql->delete( 'acl_rules', $details );  
// Update the rule with the new access value  
$this->sql->update( 'acl_rules', array( 'access' => $access ),  
    array( $this->rules as $key => $rule ) {  
        ( $details['role_id'] == $rule['role_id'] ) {  
            if ( $access == false ) {  
                unset( $this->rules[ $key ] );  
            } else {  
                $this->rules[ $key ]['access'] = $access;  
            }  
        }  
    }  
}
```

Presented by:
Group 3

Data Processing

○ Processing Objective

Building a final dataset from the Olist raw dataset to produce a clean and structured dataset that is ready for advanced machine learning modeling.

Problem

- Missing value and duplicates in raw dataset
- Outliers in numerical column
- Validity in every datasets relationship

Main Objective

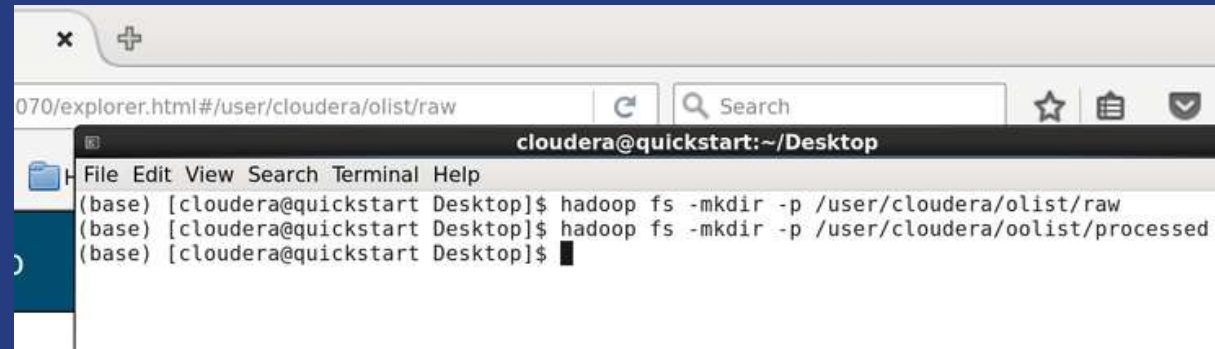
- Data Cleaning
- Aggregation
- Feature Engineering

Project Goals

- Handling missing value and duplicates
- Handling outliers in numerical column
- Referential integrity for datasets relationship checking
- Creating new column from feature engineering and aggregation for analytical purpose

Data Processing

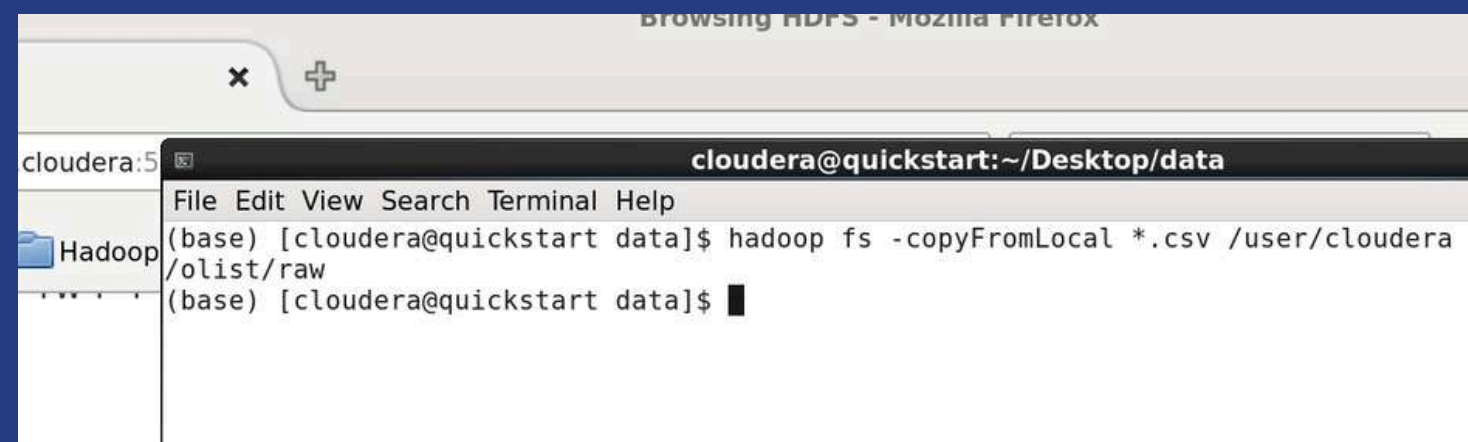
Data ingestion into storage layer (HDFS)



```
cloudera@quickstart:~/Desktop
(base) [cloudera@quickstart Desktop]$ hadoop fs -mkdir -p /user/cloudera/olist/raw
(base) [cloudera@quickstart Desktop]$ hadoop fs -mkdir -p /user/cloudera/olist/processed
(base) [cloudera@quickstart Desktop]$
```

Start from creating workplace folder inside HDFS

- Raw folder (for 9 raw dataset)
- Processed folder (for data processing result)



```
cloudera@quickstart:~/Desktop/data
(base) [cloudera@quickstart data]$ hadoop fs -copyFromLocal *.csv /user/cloudera/olist/raw
(base) [cloudera@quickstart data]$
```

Ingest 9 raw dataset from local VM filesystem to HDFS ecosystem

Data Load using PySpark

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import isnull, col, sum, count, round, countDistinct, lower
from pyspark.sql.functions import regexp_replace, length, lpad
import pandas as pd
from pyspark.sql import functions as F
from pyspark.sql.types import TimestampType

spark = SparkSession.builder \
    .appName("Olist Project") \
    .getOrCreate()
```

Import required library including pyspark to initiate spark session

```
df_orders = spark.read.csv(
    "hdfs://localhost:8020/user/cloudera/olist/raw/olist_orders_dataset.csv",
    header=True,
    inferSchema=True
)
```

Load 9 dataset from HDFS to the spark DataFrame using spark.read.csv

- header=True, used so that the first row is recognized as the name column
- inferSchema=True, used so that Spark can detect and automatically convert data types automatically

Data Processing: Data Cleaning

Missing value

```
# cek null
cek_null_orders = df_orders.select([
    sum(col(c).isNull().cast("int")).alias(c)
    for c in df_orders.columns
])

cek_null_orders.toPandas().T
```

- Done in every dataset table
- Identification: Checking the total count of nulls in each column.
- Row Dropping `.dropna()`: Removes rows missing values to prevent broken table joins

Handling missing value

```
df_orders = df_orders.dropna(subset=[
    "order_purchase_timestamp",
    "order_delivered_customer_date"
])
```

- **Done only in main dataset table i.e olist_orders**
- **Rows with nulls in important columns i.e order_purchase_timestamp, order_delivered_customer_date are removed using .dropna()**

Data Processing: Data Cleaning

Duplicates

```
total_rows_orders = df_orders.count()
unique_rows_orders = df_orders.dropDuplicates().count()

print("Total rows:", total_rows_orders)
print("Unique rows:", unique_rows_orders)
print("Duplicate rows:", total_rows_orders - unique_rows_orders)
```

- Done in every dataset table
- Identification: Checking the total count of duplicates in each column.
- Row Dropping `.dropDuplicates()`: Removes rows that have identical values in every column
- All of the datasets table doesn't have duplicates



Data Processing: Data Cleaning

○ Outlier Checking

Outlier checking on 3 numerical categories

- payment_value
- price
- freight_value

IQR Method is used to detect value that is outside normal bounds

Results of outlier checking:

- payment_value: 7.780 rows, 7.72%
- price: 8.573 rows, 7.78%
- freight_value: 11.808 rows, 10.72%

Freight value became the most extreme column with more than 10% data as outliers

```
def outlier_iqr(df, column_name):
    quantiles = df.approxQuantile(column_name, [0.25, 0.75], 0.01)
    q1 = quantiles[0]
    q3 = quantiles[1]

    iqr = q3 - q1
    lower_bound = q1 - (1.5 * iqr)
    upper_bound = q3 + (1.5 * iqr)

    total_rows = df.count()

    outliers_df = df.filter(
        (col(column_name) < lower_bound) |
        (col(column_name) > upper_bound)
    )

    outliers_count = outliers_df.count()

    outliers_percentage = (outliers_count / total_rows) * 100 if total_rows > 0 else 0

    print(f"Deteksi Outlier: Kolom '{column_name}'")
    print(f"Q1 (25%)      : {q1}")
    print(f"Q3 (75%)      : {q3}")
    print(f"IQR           : {iqr}")
    print(f"Batas Bawah : {lower_bound}")
    print(f"Batas Atas  : {upper_bound}")
    print(f"Total Data  : {total_rows} baris")
    print(f"Jml Outlier : {outliers_count} baris ({outliers_percentage:.2f}% dari total)\n")

    return outliers_df
```



Data Processing: Data Cleaning

○ Outlier Handling: Capping

Because of the large amount of outliers, it's impossible to just delete it

Instead, use capping method: values exceeding the upper limit are replaced with the upper limit value and vice versa

With capping we can maintain data referential validity and distribution without losing rows

```
def cap_outlier(df, column_name):
    quantiles = df.approxQuantile(column_name, [0.25, 0.75], 0.01)
    q1 = quantiles[0]
    q3 = quantiles[1]

    # 2. Menghitung IQR dan menentukan Batas Bawah & Atas
    iqr = q3 - q1
    lower_bound = q1 - (1.5 * iqr)
    upper_bound = q3 + (1.5 * iqr)

    print(f"[{column_name}] Proses Capping...")
    print(f"-> Nilai > {upper_bound:.2f} diubah menjadi {upper_bound:.2f}")
    print(f"-> Nilai < {lower_bound:.2f} diubah menjadi {lower_bound:.2f}\n")

    # 3. Menerapkan logika Capping
    df_capped = df.withColumn(
        column_name,
        F.when(F.col(column_name) > upper_bound, upper_bound)
        .when(F.col(column_name) < lower_bound, lower_bound)
        .otherwise(F.col(column_name))
    )

    return df_capped
```




Data Processing: Data Cleaning

```
count_payments_before = df_order_payments.count()
df_order_payments = df_order_payments.join(df_orders, on="order_id", how="left_semi")
print(f"Payments sebelum: {count_payments_before} , Sesudah: {df_order_payments.count()}")

count_items_before = df_order_items.count()
df_order_items = df_order_items.join(df_orders, on="order_id", how="left_semi")
print(f"Items sebelum: {count_items_before} , Sesudah: {df_order_items.count()}")

count_reviews_before = df_order_reviews.count()
df_order_reviews = df_order_reviews.join(df_orders, on="order_id", how="left_semi")
print(f"Reviews sebelum: {count_reviews_before} , Sesudah: {df_order_reviews.count()}")

count_customers_before = df_customers.count()
df_customers = df_customers.join(df_orders, on="customer_id", how="left_semi")
print(f"Customers sebelum: {count_customers_before} , Sesudah: {df_customers.count()}")
```

○ Referential Integrity: Left Join

After olist_order dataset has been cleaned from missing value or duplicates

Related tables must be adjusted so that no data references deleted order id.

A left semi join is conducted to retain only rows whose order_id is still valid in df_orders.

This because in the olist dataset, orders table play as a main table

Data Processing: Aggregation & Feature Engineering

Why aggregation is needed ?

- One order may contain multiple products and payments
- Raw transactional tables cannot be used directly for ML
- Different tables have different row granularities
- Aggregation prevents duplicated information during joins

○ Generated feature engineering for feature insight

- `delivery_time`
total delivery duration from purchase to arrival
- `is_late`
identifies delayed deliveries against estimated date
- `total_payment_value`
total customer payment per order
- `total_items`
number of purchased products in one transaction
- `total_price`
total order monetary value
- `total_freight`
total shipping cost per order
- `is_satisfied`
binary customer satisfaction target (1 = satisfied, 0 = dissatisfied)

Data Processing: Aggregation & Feature Engineering

○ Classification target from review table

Converting the review_score to a binary target:

1. **is_satisfied ≥ 4 become 1, which considered satisfied**
2. **is_satisfied < 4 become 0, which considered dissatisfied.**

Since an order can have more than one review line

aggregate is done again per order_id with the average, then round the result back to the final label.

```
df_reviews_feat = df_order_reviews.withColumn(
    "is_satisfied",
    F.when(F.col("review_score") >= 4, 1).otherwise(0)
)
df_reviews_agg = df_reviews_feat.groupBy("order_id").agg(
    F.avg("is_satisfied").alias("is_satisfied")
)
df_reviews_agg = df_reviews_agg.withColumn(
    "is_satisfied",
    F.when(F.col("is_satisfied") >= 0.5, 1).otherwise(0)
)
print("Target 'is_satisfied' berhasil dibuat.")
```

Data Processing: Aggregation & Feature Engineering

○ Operational feature from order table

Extract two important features from the order table:

- **delivery_time**

Delivery_time is calculated from the difference between the purchase date and the customer's receipt date

- **is_late.**

is_late indicates whether delivery is past the promised estimate.

```
#hitung selisih waktu dan keterlambatan
df_orders_feat = df_orders.withColumn(
    "delivery_time",
    F.datediff(F.col("order_delivered_customer_date"), F.col("order_purchase_timestamp"))
).withColumn(
    "is_late",
    F.when(F.col("order_delivered_customer_date") > F.col("order_estimated_delivery_date"), 1)\
    .otherwise(0)
)

#ambil kolom yg dibutuhkan aja buat master table
df_orders_clean = df_orders_feat.select("order_id", "customer_id", "delivery_time", "is_late")
print(" 'delivery_time' dan 'is_late' udh dibuat.")
```


Data Processing: Aggregation & Feature Engineering

○ Payment Aggregation

```
df_payments_agg = df_order_payments.groupby("order_id").agg(  
    F.sum("payment_value").alias("total_payment_value")  
)  
print("berhasil diagregasi.")
```

For payments, a single order can contain multiple payment methods or multiple transaction lines.

Therefore, the `payment_value` values are summed per `order_id` to create a single total payment per order.

○ Complexity Order Aggregation

```
df_items_agg = df_order_items.groupby("order_id").agg(  
    F.count("order_item_id").alias("total_items"),  
    F.sum("price").alias("total_price"),  
    F.sum("freight_value").alias("total_freight")  
)  
print("berhasil diagregasi.")
```

From the order items table, made a `total_items` feature to capture order complexity.

Additionally, `price` and `freight_value` are summed to represent the total cost and shipping cost per order.

Data Processing: Aggregation & Feature Engineering

○ Customer Location

```
df_customers_clean = df_customers.select("customer_id", "customer_state", "customer_city")
print("Fitur lokasi customer disiapkan.")
```

Maintains customer location attributes such as state and city.

This feature is essential for geographic analysis and for viewing customer patterns across regions.

○ Final integration

```
df_master = df_orders_clean \
    .join(df_customers_clean, on="customer_id", how="left") \
    .join(df_reviews_agg, on="order_id", how="left") \
    .join(df_payments_agg, on="order_id", how="left") \
    .join(df_items_agg, on="order_id", how="left")
```

Once all the features are prepared, all the tables are joined using a left join, with the order table as the primary table.

The final result is a single master dataset ready for further modeling and analysis.

Data Processing: Aggregation & Feature Engineering

Final Dataset

df_master.describe().toPandas().T

	0	1	2	3	4
summary	count	mean	stddev	min	max
order_id	96476	None	None	00010242fe8c5a6d1ba2dd792cb16214	ffffe41c64501cc87c801fd61db3f6244
customer_id	96476	None	None	00012a2ce6f8dcda20d059ce98491703	ffffa3172527f765de70084a7e53aae8
delivery_time	96476	12.497336125046644	9.555459598617153	0	210
is_late	96476	0.08112898544715784	0.27303451407502777	0	1
customer_state	96476	None	None	AC	TO
customer_city	96476	None	None	abadia dos dourados	zortea
is_satisfied	95830	0.7899405196702494	0.4073527054310869	0	1
total_payment_value	96475	133.12786815237092	93.53153374759214	9.59	1360.49
total_items	96476	1.1422115344748953	0.5388157789144618	1	21
total_price	96476	111.05609928894259	92.75187621172653	0.85	2651.25
total_freight	96476	20.33616381276175	12.50109400763337	0.7050000000000001	365.47500000000001

Load dataset to HDFS

```
df_master.coalesce(1).write.mode("append").parquet("hdfs://localhost:8020//user/cloudera/olist/processed")

#df_master.coalesce(1).write.mode("overwrite").csv("hdfs://localhost:8020//user/cloudera/olist/processed")
```


Data Analytics

○ Analytics Objective & Problem Definition

Building a predictive model to identify customer satisfaction based on transaction and delivery behavior in e-commerce services.

Problem

- Customer retention is more efficient than customer acquisition
- Customer dissatisfaction is not always detected directly
- Customers may switch to competitors

Main Objective

- Predict customer satisfaction using operational data
- Apply binary classification approach
- Categories: Satisfied (1) and Dissatisfied (0)

Project Goals

- Build a high-performance classification model
- Handle class imbalance effectively
- Identify key factors affecting customer satisfaction
- Support operational teams in improving service quality

Dataset Understanding for Analytics

The dataset used in this study was created by combining several e-commerce tables such as orders, customers, and payments. After the merging process, the data was cleaned to handle inconsistencies and remove irrelevant attributes. The preprocessing stage produced 95,830 transaction records with 11 main features used for the classification process.

Main Features

- **delivery_time** → order delivery duration
- **is_late** → delivery delay indicator
- **total_payment_value** → total customer payment value
- **total_price** → total transaction price
- **total_items** → total number of purchased items

Target Variable

- **is_satisfied** was used as the label representing customer satisfaction level.

Data Distribution

- Data Distribution
- Satisfied customers: 75,701 transactions (79%)
- Dissatisfied customers: 20,129 transactions (21%)

Challenge & Solution

- The dataset experienced class imbalance because satisfied customers dominated the data.
- To address this issue, the study applied a class weighting approach to ensure the model could recognize both customer categories more effectively.



Descriptive Analytics Implementation

This descriptive study statistically demonstrates that delivery performance and order complexity are directly correlated with customer satisfaction indicators.

Characteristics & Imbalanced Data

- **Target Variable (`is_satisfied`):** Significant class imbalance (the majority of consumers are satisfied, while a minority are dissatisfied).
- **Model Solution:** Use the `auto_class_weights="Balanced"` parameter in the algorithm to address class imbalance.

Preliminary Findings & Logistics

- Significant differences in logistics performance were found for the variables **"is_late"** (delays) and **"delivery_time"** (delivery duration).
- These two factors were identified as the primary causes of customer complaints.

Multidimensional Correlation & Data Transformation

- **The analysis covers operational and financial aspects:** `total_items`, `customer_state_idx`, and `total_payment_value`.
- **Data Transformation:** The `total_payment_value` variable has a very wide distribution range, so a log transformation was applied to stabilize the data variance before feeding it into the model.

Classification Model Development

Model Selection & Rationale

Primary Model: CatBoost Classifier

- Leverages built-in Ordered Boosting to robustly prevent overfitting.
- Utilizes native Target Encoding for high-cardinality categorical data (customer_state).

Comparator Model: Random Forest Classifier

Acts as a baseline benchmark utilizing ensemble bagging trees.

Orchestrated via PySpark MLlib and integrated Python ML pipelines.

```
def make_pipeline():  
    """Builds a new Pipeline to guarantee consistency"""  
    rf = RandomForestClassifier(**RF_PARAMS)  
    return Pipeline(stages=[fitted_indexer, assembler, rf])
```

```
fold_pipeline = make_pipeline()  
fold_model    = fold_pipeline.fit(spark_fold_train)  
fold_preds    = fold_model.transform(spark_fold_val)
```

```
final_pipeline = make_pipeline()  
final_model    = final_pipeline.fit(spark_train_final)
```

```
test_pool = Pool(  
    X_test,  
    cat_features=cat_cols
```

```
)  
train_pool = Pool(  
    X_tr,  
    y_tr,  
    cat_features=cat_cols
```

Feature Engineering & Pipeline Integration

Categorical Encoding: StringIndexer maps geographic data (customer_state) into numeric indices.

```
state_indexer = StringIndexer(  
    inputCol="customer_state",  
    outputCol="customer_state_idx",  
    handleInvalid="keep"  
)
```

Feature Assembly: VectorAssembler consolidates features (delivery time, delay status, transaction values, encoded state) into a unified feature vector.

```
assembler = VectorAssembler(  
    inputCols=FEATURE_COLS,  
    outputCol="features"  
)
```

Pipeline Consistency: Strict execution flow ensuring identical, non-corrupted data transformations across both training and testing phases.

Classification Model Development

Training Setup & Validation Strategy

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    stratify=y,
    random_state=42
```

Deterministic Split: Data is partitioned into 80% Training / 20% Testing (Random Seed: 42 for strict repeatability).

```
skf = StratifiedKFold(
    n_splits=10,
    shuffle=True,
    random_state=42
)
```

Robust Validation: Implemented a 10-Fold Cross-Validation (CV) routine.

- Models are re-instantiated at each fold to prevent state leakage.
- Guarantees stable, objective, and unbiased evaluation metrics across different data partitions.

Handling Class Imbalance & Overlapping Bottleneck

The Imbalance Reality: Severe class distribution with 79% Satisfied vs. 21% Unsatisfied customers.

is_satisfied	total
0	20130
1	75700

```
cb_model_final = CatBoostClassifier(
    iterations=average_best_iteration,
    learning_rate=0.03,
    depth=8,
    eval_metric="AUC",
    auto_class_weights="Balanced",
    l2_leaf_reg=5,
    random_seed=42,
    task_type="CPU"
)
```

Strategic Countermeasures:

- **CatBoost:** Activated `auto_class_weights="Balanced"` to inherently penalize majority class bias.
- **Random Forest:** Enhanced via ADASYN oversampling combined with Gaussian Noise augmentation to dynamically synthesize minority samples.

```
ada_fold = ADASYN(random_state=SEED)
X_f_res, y_f_res = ada_fold.fit_resample(X_f_noisy, df_f_train_raw[TARGET_COL])
```

Current Bottleneck: Stagnant accuracy boundaries due to heavy feature/class overlapping, posing a structural challenge for the classifiers.

Model Evaluation & Performance Analysis

○ Evaluation Metrics & Strategy

- **Comprehensive Evaluation:** Both models are evaluated using Accuracy, F1-Score, Precision, Recall, and AUC-ROC.
- **F1-Score:** Balances Precision and Recall, providing a reliable and fair assessment despite the severe class imbalance and heavy data overlapping.
- **Confusion Matrix:** Implemented to visualize exact classification errors and demonstrate how well the model predicts actual labels.
- **AUC-ROC:** Ensures and measures the model's overall ability to distinguish between satisfied and dissatisfied customers across all possible classification thresholds.

○ Feature Importance & Business Insights

- **Top 2 Dominant Drivers:**

1
delivery_time
Importance Score: 31.11

2
is_late
(Importance Score: 27.79)

- **Core Strategic Insight:** Logistics reliability and delivery schedules hold significantly more influence over customer satisfaction than price variations or payment methods.
- **Business Recommendation:** Operational planning must pivot heavily toward enhancing delivery efficiency and improving the accuracy of estimated arrival times (ETA) to mitigate future customer dissatisfaction.



Feature Importance & Business Insights

Top Predictors (CatBoost Model Results)

- Based on the weights of the CatBoost Classifier model, the **Logistics Indicator** is the most **critical factor** in determining customer satisfaction.
- The most influential features, in order: is_late & delivery_time → total_items → customer_state_idx.

Operational Impact & Fulfillment Risk

- **Logistics:** Timeliness is key. Even a single delay (is_late = 1) immediately shifts the model's prediction toward customer dissatisfaction (dissatisfied).
- **Complexity & Geography:** Orders with many items (total_items) increase the risk of operational failure. Additionally, logistics infrastructure across Brazilian states (customer_state_idx) causes variations in wait times.
- **Financial:** Although the weight of total_payment_value is lower, this feature is important for assessing the sensitivity of high-value customers to service quality.

Strategic Recommendations

- Companies must make supply chain efficiency a top priority.
- Conduct regular mapping of distribution areas to gradually reduce delays.



Customer Satisfaction Dashboard

Brazilian E-Commerce - ML Classification Insights

Executive Summary

Delivery & Late Analysis

Regional & Order Complexi...

95.83K

Total Orders

79%

Satisfaction Rate

8%

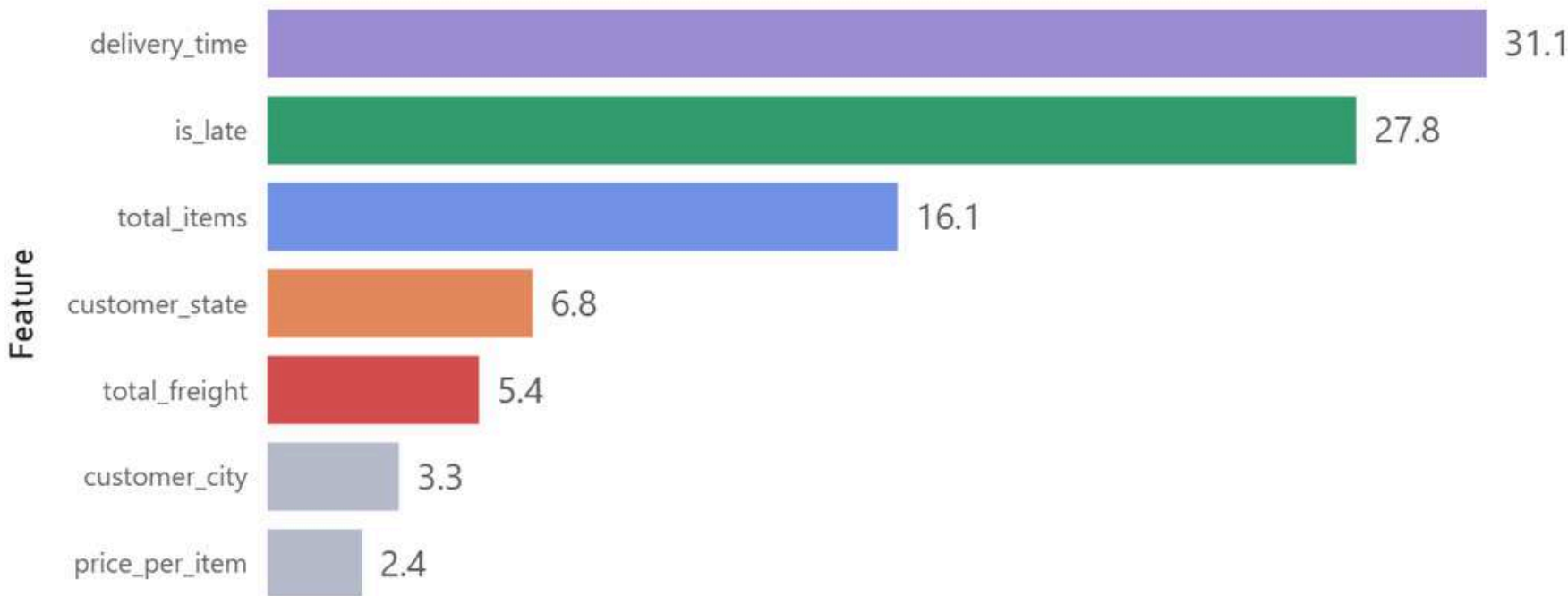
Late Delivery Rate

12.46

Avg Delivery Time

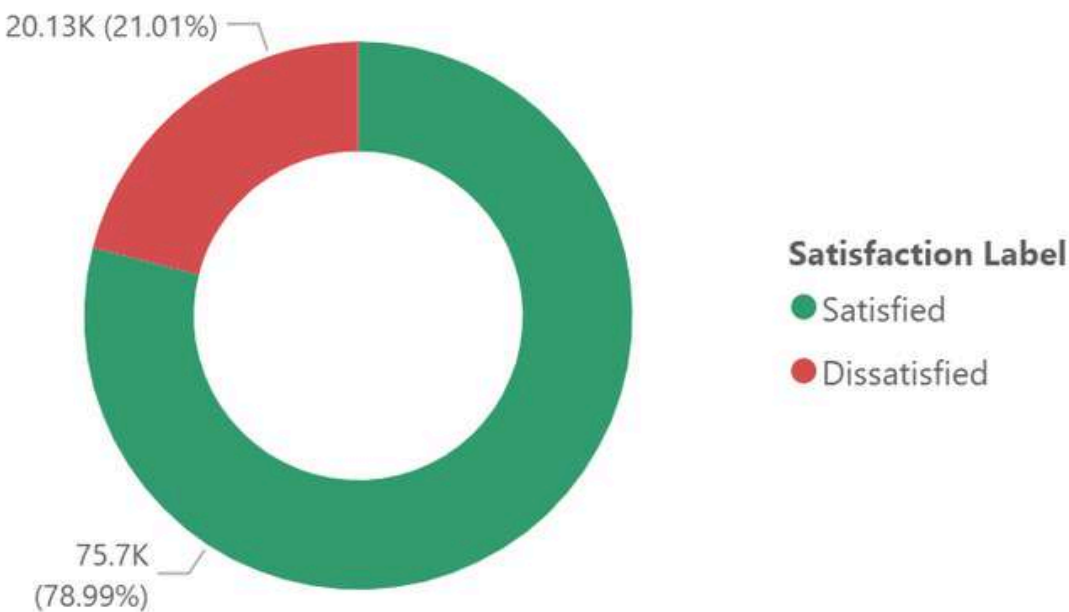
What drives customer dissatisfaction?

Feature importance — ML model



Sum of Importance

Overall satisfaction breakdown





Delivery & Late Delivey Analysis

Logistics as the primary satisfaction driver

Executive Summary

Delivery & Late Analysis

Regional & Order Complexity..

Satisfaction Rate by Delivery Bucket



customer_state

All

Late deliveries drive dissatisfaction

Satisfaction Label ● Dissatisfied ● Satisfied

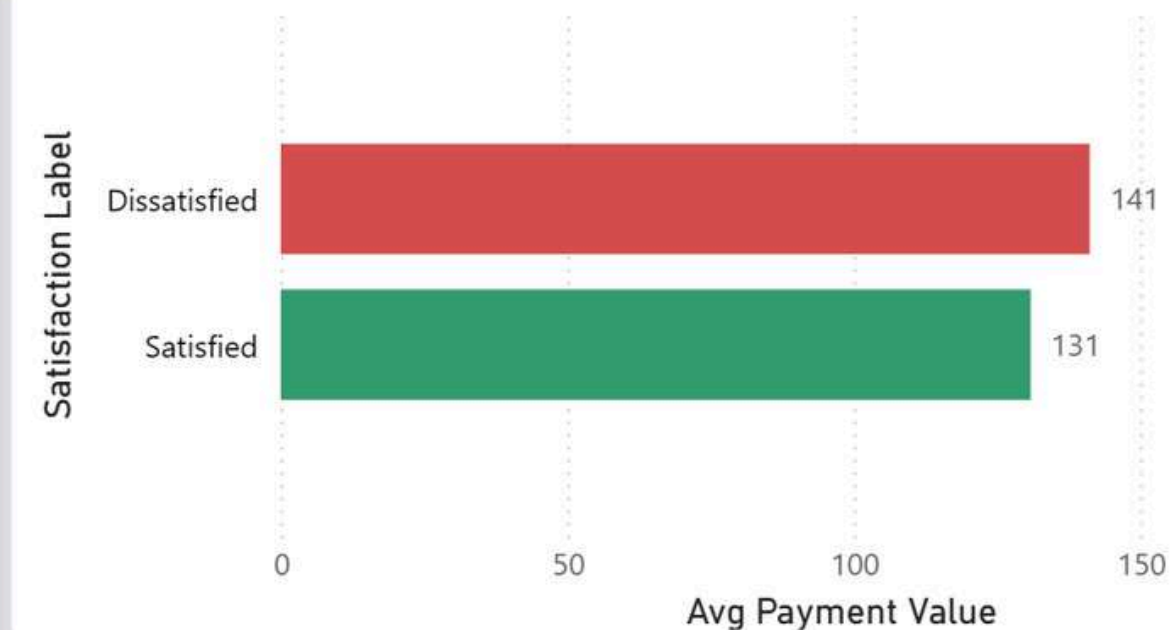


High-value orders + long delivery = risk zone

Satisfaction Label ● Dissatisfied ● Satisfied



Disssatisfied customers have higher order values





Visualization



Customer Satisfaction Dashboard

Brazilian E-Commerce - ML Classification Insights

Executive Summary

Delivery & Late Analysis

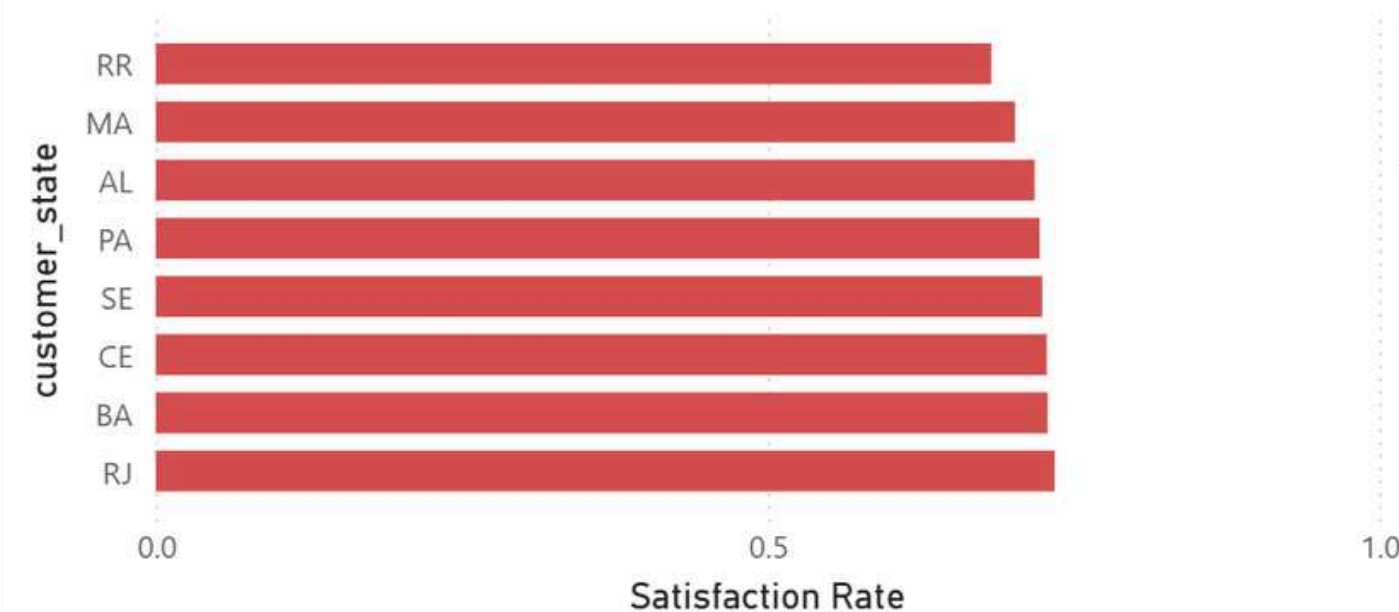
Regional & Order Complexi...

Satisfaction rate across Brazilian states

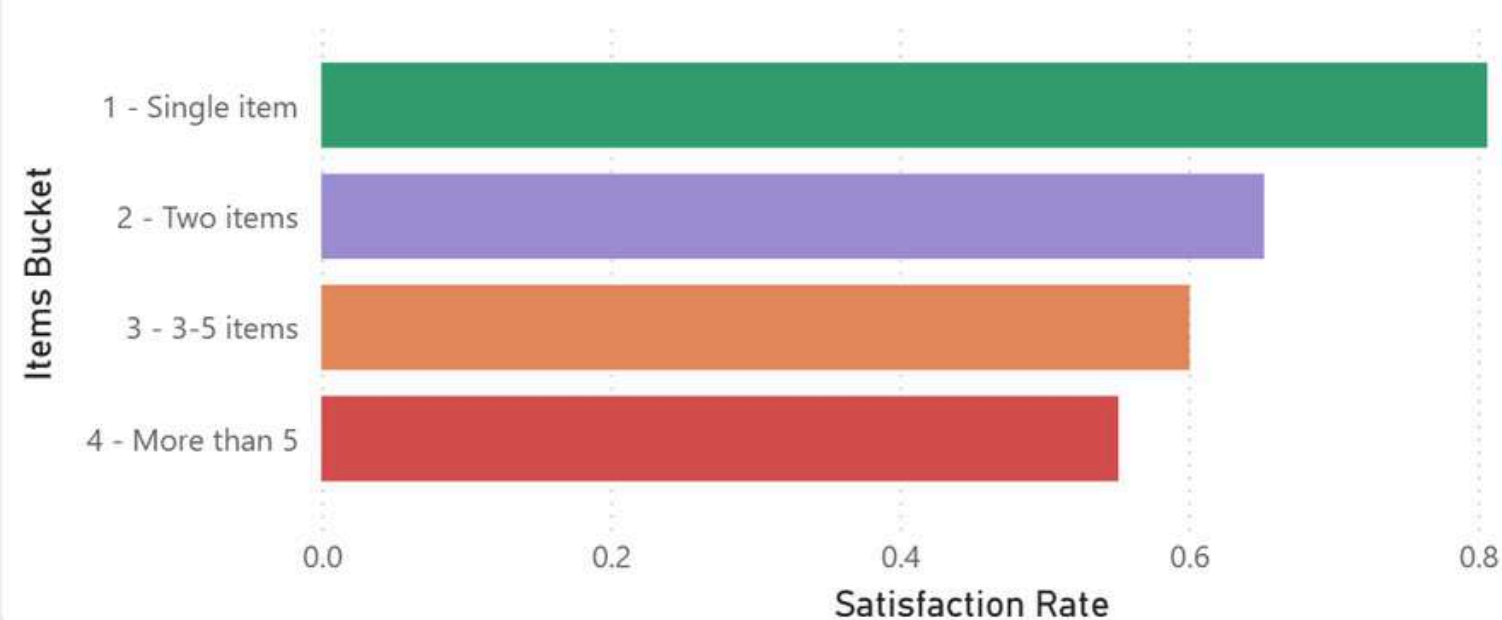
Filled map — color saturation = satisfaction rate



Which states need logistics improvement?



Multi-item orders are harder to satisfy



Avg Delivery Time by customer_state



VIZUALIZATION ANALYSIS

Root Cause & ML Model Results

The classification model using Machine Learning reveals that the main factors contributing to customer dissatisfaction are delivery time (31.1) and is_late (27.8). The complexity of order volume (total items: 16.1) stands out as the third significant element, where customer satisfaction drops sharply (approximately 55%) for orders with multiple items (over 5 products) due to extended periods required for packaging and consolidation.

Regional Correlation & Delay Impact

A significant variation exists between regions, particularly in remote areas like Roraima (RR) and Maranhão (MA), which show the longest delivery times (near 30 days) as well as the least satisfaction.

A delivery time that exceeds 21 days acts as a crucial limit for maintaining customer satisfaction; furthermore, occurrences of delays in delivery lead to a noticeable increase in dissatisfaction rates, which can rise as high as 65.32%.

Business Risks & Strategic Recommendations

Analyses of finances indicate a considerable risk of churning high-value customers, with the average expenditure of unhappy customers (141) being markedly greater than that of content customers (131).

This trend illustrates a direct relationship between higher transaction amounts and elevated service expectations; as a result, inefficiencies in logistics appear to drive away the most valuable customer group.

It is advisable for the business to develop decentralized warehousing systems in vital areas (RR & MA) to reduce shipping times, instead of implementing strategies aimed at lowering prices, which the ML model indicated would have a negligible impact (2.4).



Phase 4 Result and Discussion

```
source_id' => $resource_id  
if ( ! $resource_exists( $resource_details['id'] ) ) {  
    $access == false ) {  
        // Remove the rule as there is currently  
        $details['access'] = !$access;  
        $this->_sql->delete( 'acl_rules', $details  
    ) {  
        // Update the rule with the new access value  
        $this->_sql->update( 'acl_rules', array( 'ac  
    foreach( $this->rules as $key=>$rule ) {  
        if ( $details['role_id'] == $rule['role_id'] )  
        if ( $access == false ) {  
            unset( $this->rules[ $key ] );  
        } else {  
            $this->rules[ $key ]['access']
```

Presented by:
Group 3



Result & Business Insight

○ Key Insights & Delivery Performance Analysis

Delivery performance is the primary driver of customer satisfaction, where on-time arrivals yield an 82.85% satisfaction rate, while delays cause it to plummet to 34.68%. Furthermore, prompt deliveries under 7 days achieve nearly 90% satisfaction, compared to just 33% for wait times exceeding 30 days. This downward trend is particularly evident in specific regional bottlenecks—namely RR, MA, AL, PA, and SE—where transit times average 20 to 30 days. Order complexity also plays a critical role; single-item purchases maintain an 81% satisfaction rate, whereas larger orders containing more than five items drop to 55% due to more complicated fulfillment and packaging challenges.

○ Strategic Business Recommendations

To reverse these trends and boost long-term customer retention, the business must strategically overhaul its logistics. Key initiatives should include establishing regional fulfillment centers in low-satisfaction states, partnering with local couriers to accelerate last-mile delivery, and pre-positioning popular inventory closer to high-demand areas. Additionally, the handling of complex, multi-item orders can be significantly improved by deploying automated picking systems, introducing stricter pre-shipment quality control checks, and optimizing routing management to prevent delays.



Challenges & Limitation

- **Data Coverage and Referential Integrity Issues** : There are several missing values in the Olist dataset. To maintain data integrity and accuracy for analysis, missing zip codes between the seller and geolocation tables must be resolved to prevent incomplete geographic mapping, while a regex tool is required to remove disguised store names from customer reviews before NLP processing.
- **High Numbers of Financial Outliers** : Transaction values in the financial table which is olist_order_payments table show significantly more outliers than expected.
- **Class Imbalance and Low Data Separability** : Despite shifting the system's focus to dissatisfied customers, the upper score limit remains firmly stuck at 81.98% because their purchasing behavior and data log patterns overlap and look virtually identical to those of satisfied customers.
- **Infrastructure and Batch Processing Constraints** : Because the system processes sales data in sequential stages using a single-node Hadoop system and Google Colab, it cannot capture user changes in real time, making it inherently inadequate for immediate detection of suspicious activity.

Future Improvement

Several technical and analytical challenges were encountered, shaping the boundaries of the current model's capabilities:

Real-Time Streaming Integration

Current Limitation

- Current system uses batch processing approach
- Data is processed after historical data is collected
- Insights cannot yet support live monitoring

Proposed Development

- Implement real-time streaming architecture
- Integrate Apache Kafka for continuous data streaming
- Utilize Apache Spark Structured Streaming for distributed real-time processing

Expected Benefits

- Detect delivery delays earlier
- Monitor customer transaction patterns dynamically
- Improve logistics and service quality monitoring
- Support faster and more responsive operational decisions

Advanced Analytics Enhancement

Future Model Exploration

- **Alternative Algorithms:** Benchmark against models natively robust to high data overlap and imbalance, such as SVM (RBF Kernel), TabPFN, and Neural Networks.
- **Performance Baseline:** Compare these new architectures directly against the current best-performing baseline (CatBoost: 81.98% accuracy).

Feature & Data Engineering Enhancements

- **Advanced Feature Addition:** Introduce more representative attributes, focusing heavily on customer behavior metrics
- **Target Labeling Strategy:** Re-evaluate and alter the current data aggregation methodology to sharpen the decision boundary and resolve the low separability problem causing the current accuracy stagnation.

Distributed Infrastructure Scalability

Migration to a Multi-Node Hadoop Cluster

- **Future Plans:** Transition from a single-node environment to a multi-node Hadoop cluster architecture to handle the exponential growth in e-commerce data volume.
- **HDFS Storage:** Data will be automatically replicated across multiple worker nodes.
- **Benefits:** Improves data availability and fault tolerance. The system remains stable even in the event of hardware failure because the resource manager will redirect tasks to other nodes.

Acceleration with Distributed Spark Processing

- **In-Memory Parallel Processing:** Integration of Apache Spark into a multi-node architecture to accelerate the execution time of parallel data processing directly in memory.
- **Optimizing Heavy Computational Workloads:** Accelerate intensive processes such as CatBoost algorithm iterations and repeated 10-fold cross-validation evaluations.

Conclusion



This project demonstrates that big data technology using HDFS and Apache Spark effectively optimizes the storage, distributed processing, and ETL transformation of large-scale e-commerce data. At the analytics stage, the CatBoost model achieved an 81.98% accuracy in predicting customer satisfaction, revealing that logistical factors—specifically delivery_time and is_late, are the primary drivers of user experience rather than pricing or payment methods. Ultimately, the integration of this machine learning model with an interactive monitoring dashboard successfully pinpoints supply chain bottlenecks across regions, proving that a data-driven approach provides a solid foundation for companies to enhance operational efficiency, maximize delivery reliability, and sustainably elevate the end-customer experience.

AI Declaration

AI Role in Data Processing

- **Data Cleaning:** Assisted in writing automated syntax for numeric outlier detection and data capping methods.
- **Pipeline Debugging:** Provided instant code references and solutions to resolve syntax errors during feature engineering and aggregation.
- **Data Deployment:** Guided the seamless export of final datasets from Apache Spark notebooks into the HDFS ecosystem and Cloudera VM.

AI Integration in Model Development

- **Pipeline Setup:** Supplied PySpark MLlib code templates to build clean modeling pipelines and construct the 10-Fold Cross-Validation loop.
- **Imbalance Strategy:** Provided implementation references for class weighting to handle the severe 79:21 class imbalance.
- **Algorithm Benchmarking:** Offered experimental configuration setups to compare Random Forest and CatBoost performances.

AI in Visualization & Team Validation

- **Dashboard Design:** Recommended color themes, visual layouts, and auxiliary columns to enhance the PowerBI dashboard structure.
- **Skill Adaptation:** Provided step-by-step guidance to quickly master and implement unfamiliar, advanced features in PowerBI.
- **Human Oversight:** The team independently cross-checked and validated all evaluations, metrics, and interpretations to ensure absolute academic integrity.



Thank You Very Much For the Attention

Presented by:
Group 3

